

实验五：鸿蒙多端应用开发（Maul 技术栈）

一、实验基本信息

项目	内容
实验编号	d20301035105、d20301009205
学时分配	4 学时
实验类型	验证型实验
每组人数	6 人（分组协作完成）
对应课程目标	课程目标 3：掌握鸿蒙多端开发、Maul 组件使用及分布式数据同步能力
使用技术栈	Maul + ArkTS + 鸿蒙 Next 核心 API + 传感器 API + 分布式 KVStore

二、实验目的

- 掌握鸿蒙多端应用的核心开发流程与技术要点，熟悉 Maul 组件库的使用方法。
- 掌握 ArkTS 声明式 UI 开发，实现手机、平板多端自适应布局，理解响应式设计原理。
- 掌握鸿蒙传感器 API 调用，实现光线、陀螺仪数据的采集与展示。
- 掌握鸿蒙分布式数据同步机制，实现跨设备数据实时同步。
- 理解鸿蒙多端开发的技术优势，能够独立完成简单多端应用的开发、调试与验证。
- 能够结合 Maul 技术栈，优化多端应用的开发效率与 UI 适配效果。

三、实验环境

3.1 鸿蒙开发环境

- 开发工具：DevEco Studio（最新版）
- 开发语言：ArkTS
- UI 框架：Maul 组件库
- 运行环境：鸿蒙 Next 模拟器（手机、平板）

3.2 核心依赖与工具

- Maul 组件库：用于快速构建多端适配 UI，简化布局与间距适配
- 传感器 API：鸿蒙系统传感器模块，实现光线、陀螺仪数据采集
- 分布式 KVStore：实现跨设备数据同步与存储
- TRAE AI 工具：辅助生成 ArkTS 代码，提升开发效率

四、实验内容

4.1 核心任务

基于 Maul 技术栈开发“智慧天气”多端应用，实现以下功能，达成多端统一运行效果：

- 对接模拟天气数据，展示城市、温度、天气状况、空气质量等核心信息。
- 实现手机、平板多端自适应布局，适配不同屏幕尺寸与显示比例。
- 集成光线传感器、陀螺仪，实现环境数据采集与实时展示。
- 实现分布式数据同步，验证跨设备数据实时更新功能。
- 完成技术分析报告，总结多端适配与硬件集成的关键技术要点。

4.2 实战案例

开发“智慧天气”应用，基于 Maul 组件构建 UI 界面，集成传感器与分布式能力，实现一套代码、多端运行，体验鸿蒙多端开发的便捷性，验证 Maul 技术栈在多端适配中的优势。

4.3 开发任务清单

任务阶段	主要内容	关键技术点
天气应用开发	ArkTS 声明式 UI 开发，Maul 组件适配	@State、数据绑定、Maul 基础组件

多端适配	手机/平板布局自适应，屏幕尺寸适配	断点系统、媒体查询、Maul 适配方法
传感器集成	光线、陀螺仪数据采集与展示	传感器 API 调用、数据实时监听
分布式同步	跨设备数据同步，多端数据一致性	分布式 KVStore、数据监听与更新

五、实验步骤

5.1 天气应用开发与多端适配（Maul 版）

- 创建项目：**打开 DevEco Studio，新建 Empty Ability 项目，选择 ArkTS 语言，命名为 WeatherApp，集成 Maul 组件库。
- 配置依赖：**确认 Maul 组件库已导入，无需额外配置，直接调用 MContainer、MColumn 等 Maul 组件。
- AI 辅助编码：**使用 TRAE 工具，输入需求“ArkTS + Maul 天气应用页面，声明式 UI，城市、温度、天气状况展示”，生成基础代码并优化。
- 核心代码实现：**编写天气首页代码，实现城市、温度、天气状况的展示，代码如下：

```
@Entry
@Component
struct Index {
  @State cityName: string = '合肥';
  @State temperature: number = 22;
  @State weather: string = '晴';
  @State aqi: number = 45;

  build() {
    // Maul 容器，自动适配安全区和多端间距
    MContainer() {
      MColumn([
        Text(this.cityName)
          .fontSize(24)
          .fontWeight(FontWeight.Bold),
        Text(`${this.temperature}°C`)
```

```

        .fontSize(60)
        .fontWeight(FontWeight.Lighter),
        Text(this.weather)
        .fontSize(20),
        MRow([
            Text('空气质量: '),
            Text(`${this.aqi} (优)`)
                .fontColor('#00ff00')
        ])
    ])
    .alignItems(HorizontalAlign.Center)
}
.width('100%')
.height('100%')
.backgroundColor('#f5f5f5')
}
}

```

5. **多端适配实现**：添加设备判断与布局切换逻辑，实现手机/平板自适应，代码如下：

```

@State currentBreakpoint: string = 'md';
@State isPhone: boolean = true;

aboutToAppear() {
    // Maui 内置方法判断设备类型
    this.isPhone = MauiUI.isPhone();
    let windowWidth =
window.getWindow().getWindowProperties().windowWidth;
    this.currentBreakpoint = windowWidth < 600 ? 'sm' : 'lg';
}

build() {
    MContainer() {
        if (this.isPhone) {
            // 手机布局：单列居中
            MColumn([this.phoneLayout()])
                .alignItems(HorizontalAlign.Center)
        } else {
            // 平板布局：双列均分
            MRow([
                this.tabletLeftLayout(),
                this.tabletRightLayout()
            ]).width('100%').spacing(20)
        }
    }
}
}

```

```

        .fill()
        .bgColor('#f5f5f5')
    }

    // 手机布局组件
    @Builder phoneLayout() {
        Column() {
            Text(this.cityName).fontSize(24)
            Text(`${this.temperature}°C`).fontSize(50)
        }
    }

    // 平板左侧布局
    @Builder tabletLeftLayout() {
        Column() {
            Text(this.cityName).fontSize(32).fontWeight(FontWeight.Bold)
            Text(`${this.temperature}°C`).fontSize(80)
        }
        .width('40%')
        .alignItems(HorizontalAlign.Center)
    }

    // 平板右侧布局
    @Builder tabletRightLayout() {
        Column() {
            Text(this.weather).fontSize(28)
            Text(`空气质量: ${this.aqi} (优)`).fontSize(22)
        }
        .width('60%')
        .padding({ left: 30 })
    }

```

5.2 传感器集成开发

1. **光线传感器开发**：导入 sensor 模块，实现环境亮度采集与展示，代码如下：

```

import sensor from '@ohos.sensor';

@Entry
@Component
struct LightSensorPage {
    @State lightIntensity: number = 0;

    aboutToAppear() {
        // 监听光线传感器数据
    }
}

```

```

        sensor.on(sensor.SensorType.SENSOR_TYPE_ID_LIGHT, (data) => {
            this.lightIntensity = data.light;
        });
    }

    aboutToDisappear() {
        // 取消监听，避免资源泄漏
        sensor.off(sensor.SensorType.SENSOR_TYPE_ID_LIGHT);
    }

    build() {
        MContainer() {
            MText("光线传感器数据")
                .fontSize(22)
                .fontWeight(FontWeight.Bold),
            MCard() {
                Text(`当前环境亮度: ${this.lightIntensity.toFixed(0)} lux`)
                    .fontSize(18)
            }
            .bgColor('#e3f2fd')
        }
        .fill()
        .padding(20)
    }
}

```

2. 陀螺仪传感器开发：实现设备姿态采集，代码如下：

```

import sensor from '@ohos.sensor';

@Entry
@Component
struct GyroscopePage {
    @State rotateX: number = 0;
    @State rotateY: number = 0;
    @State rotateZ: number = 0;

    aboutToAppear() {
        sensor.on(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE, (data) => {
            this.rotateX = data.x;
            this.rotateY = data.y;
            this.rotateZ = data.z;
        });
    }
}

```

```

aboutToDisappear() {
    sensor.off(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE);
}

build() {
    MContainer() {
        MText("陀螺仪数据")
            .fontSize(22)
            .fontWeight(FontWeight.Bold),
        MRow([
            MCard() { Text(`X 轴: ${this.rotateX.toFixed(2)}`) },
            MCard() { Text(`Y 轴: ${this.rotateY.toFixed(2)}`) },
            MCard() { Text(`Z 轴: ${this.rotateZ.toFixed(2)}`) }
        ])
        .spacing(20)
    }
    .fill()
    .padding(20)
}
}

```

3. 权限配置：在 module.json5 中添加传感器与分布式权限，确保功能正常运行：

```

"requestPermissions": [
    { "name": "ohos.permission.INTERNET" },
    { "name": "ohos.permission.SENSOR" },
    { "name": "ohos.permission.DISTRIBUTED_DATASYNC" }
]

```

5.3 分布式数据同步开发

1. 初始化分布式 **KVStore**，实现跨设备数据同步，代码如下：

```

import distributedData from '@ohos.data.distributedData';

@Entry
@Component
struct DistributedSyncPage {
    @State syncCity: string = '合肥';
    @State syncTemp: number = 22;

    // 初始化分布式存储
    async initKVStore() {
        try {
            const KVManager = distributedData.createKVManager({
                bundleName: 'com.example.weatherapp',
                kvStoreType: distributedData.KVStoreType.SINGLE_VERSION
            });

```

```

    });
    const kvStore = await KVManager.getKVStore('weatherStore');
    // 写入初始数据
    await kvStore.put('city', this.syncCity);
    await kvStore.put('temperature', this.syncTemp.toString());
    // 监听数据变化（跨设备同步）
    kvStore.on('change', (data) => {
        this.syncCity = data.city;
        this.syncTemp = Number(data.temperature);
    });
} catch (e) {
    console.error('分布式 KVStore 初始化失败: ', e);
}
}

aboutToAppear() {
    this.initKVStore();
}

build() {
    MContainer() {
        MText("分布式数据同步演示")
            .fontSize(22)
            .fontWeight(FontWeight.Bold),
        MCard() {
            Column() {
                Text(`同步城市: ${this.syncCity}`)
                Text(`同步温度: ${this.syncTemp}°C`)
            }
            .padding(30)
        }
        .bgColor('#f3e5f5')
    }
    .fill()
    .padding(20)
}
}

```

2. **同步验证**：启动两个模拟器（手机、平板），登录同一华为账号，验证数据实时同步效果，确保修改一端数据后，另一端自动更新。

六、实验结果

6.1 成功标准验证

1. 功能完整性：天气应用可正常展示城市、温度、天气状况、空气质量，无功能异常。
2. 多端适配：手机、平板布局切换正常，无布局错乱、字体异常，Maul 组件适配效果达标。
3. 传感器功能：光线、陀螺仪数据采集正常，实时更新，无数据丢失或延迟。
4. 分布式同步：跨设备数据实时同步，修改数据后两端同步更新，无错误。
5. 代码可运行：所有核心代码可正常编译、运行，无语法错误，注释规范。
6. 报告完整性：完成技术分析报告，符合模板要求，内容详实。

6.2 界面特点

1. 手机端：单列居中布局，聚焦核心天气信息，字号适中，操作简洁，适配小屏幕显示。
2. 平板端：双列布局，左侧展示核心信息，右侧补充详细内容，充分利用大屏空间。
3. Maul 组件优势：无需手动调整间距、安全区，组件自带适配效果，大幅提升开发效率。

6.3 实验界面截图展示

本次实验通过 DevEco Studio 模拟器完成全功能验证，分别对手机端、平板端界面，以及传感器、分布式同步功能进行截图记录，截图占位如下，便于后续插入实验实际运行效果截图：

图 1 天气应用主界面（手机端）



图 2 天气应用主界面（平板端）



图 3 光线传感器数据展示界面



图 4 陀螺仪传感器数据展示界面



6.4 技术分析报告（模板）

markdown

多端适配与硬件集成技术分析报告（Maul 技术栈）

1. 多端适配分析

- 手机布局特点：单列居中布局，聚焦城市、温度等核心信息，字号适配小屏幕，操作流程简洁，符合手机用户使用习惯。
- 平板布局特点：双列均分布局，左侧展示核心温度、城市信息，右侧补充天气状况、空气质量，充分利用大屏优势，提升信息展示效率。
- 适配实现：通过 `MaulUI.isPhone()` 方法判断设备类型，结合屏幕宽度设置断点（`sm/lg`），实现布局自动切换，无需额外编写多套代码。
- Maul 组件优势：MContainer、MColumn、MCard 等组件自带多端适配能力，无需手动配置间距、安全区，简化开发流程，提升开发效率。

2. 传感器集成分析

- 光线传感器：用于采集环境亮度，可联动界面亮度自动调节，提升用户体验，采集延迟低、数据准确，需注意销毁时取消监听，避免资源泄漏。
- 陀螺仪传感器：用于采集设备姿态，可拓展实现体感交互（如旋转设备切换城市），数据采集实时性强，适配鸿蒙传感器 API 调用规范。
- 权限配置：需在 `module.json5` 中配置 `SENSOR`、`DISTRIBUTED_DATASYNC` 权限，否则无法正常获取传感器数据与实现跨设备同步。

3. 分布式能力分析

- 原理：基于鸿蒙分布式 `KVStore`，采用键值对存储方式，依赖同一华为账号与局域网环境，实现跨设备数据实时同步。
- 实现：通过创建 `KVManager`、监听数据变化，实现城市、温度等核心数据的同步更新，确保多端数据一致性。
- 注意事项：需确保多设备登录同一华为账号、处于同一局域网，否则同步功能无法正常生效。

4. 技术选型建议

1. UI 开发：优先使用 `MauI` 组件库，简化多端适配代码，保证 UI 风格统一，提升开发效率。
2. 状态管理：使用 `@State` 实现简单状态管理，复杂场景可结合 `@Link`、`@Provide` 等装饰器，确保数据与 UI 联动。
3. 传感器调用：统一封装传感器监听与销毁方法，避免资源泄漏，提升代码可维护性。
4. 分布式开发：提前确认设备环境（账号、网络），做好异常处理，避免同步失败导致的功能异常。

5. 问题与优化方向

- 常见问题：传感器权限配置遗漏、分布式同步失败、多端布局错乱。
- 优化方向：完善异常处理逻辑，增加加载状态、错误提示；优化 `MauI` 组件布局细节，提升多端适配兼容性。

6.5 UML 模型设计与解析

为更清晰地展示本项目整体架构、类结构与交互流程，本次实验使用 UML 建模工具完成架构图、类图、顺序图的设计，以下为各图表内容与详细解析。

6.5.1 系统架构图

PlantUML 代码

```
@startuml
    鸿蒙多端天气应用架构图
```

title 鸿蒙多端天气应用（Maul 技术栈）架构

```
rectangle 表示层【UI 层】 {  
    component 天气首页  
    component 多端适配布局  
    component 传感器展示页面  
    component 分布式同步页面  
}
```

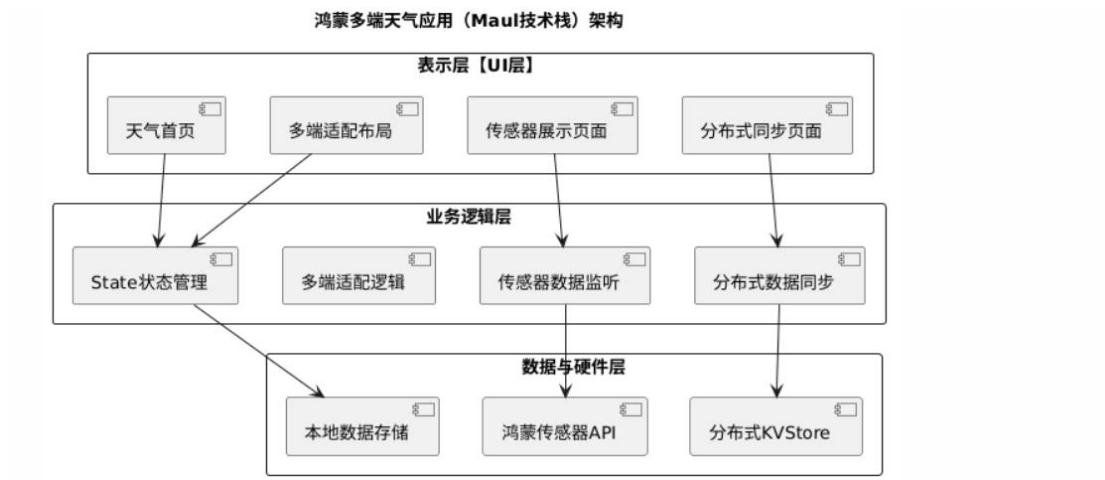
```
rectangle 业务逻辑层 {  
    component State 状态管理  
    component 多端适配逻辑  
    component 传感器数据监听  
    component 分布式数据同步  
}
```

```
rectangle 数据与硬件层 {  
    component 鸿蒙传感器 API  
    component 分布式 KVStore  
    component 本地数据存储  
}
```

天气首页 --> State 状态管理
多端适配布局 --> State 状态管理
传感器展示页面 --> 传感器数据监听
分布式同步页面 --> 分布式数据同步

State 状态管理 --> 本地数据存储
传感器数据监听 --> 鸿蒙传感器 API
分布式数据同步 --> 分布式 KVStore

@enduml



架构图解析

本项目采用三层架构设计，层次清晰、职责明确，便于维护与扩展。表示层负责用户界面展示与交互，基于 **Maul** 组件快速构建；业务逻辑层负责核心逻辑处理，是连接 UI 与底层数据的核心枢纽；数据与硬件层负责与鸿蒙系统能力交互，为上层提供稳定的数据与硬件支撑，整体体现了解耦、复用、可扩展的软件工程思想。

6.5.2 项目类图

PlantUML 代码

```
puml
@startuml 鸿蒙天气应用类图
class Index {
    - cityName: string
    - temperature: number
    - weather: string
    - aqi: number
    - isPhone: boolean
    + build()
    + phoneLayout()
    + tabletLeftLayout()
    + tabletRightLayout()
}

class LightSensorPage {
    - lightIntensity: number
    + aboutToAppear()
    + aboutToDisappear()
    + build()
}
```

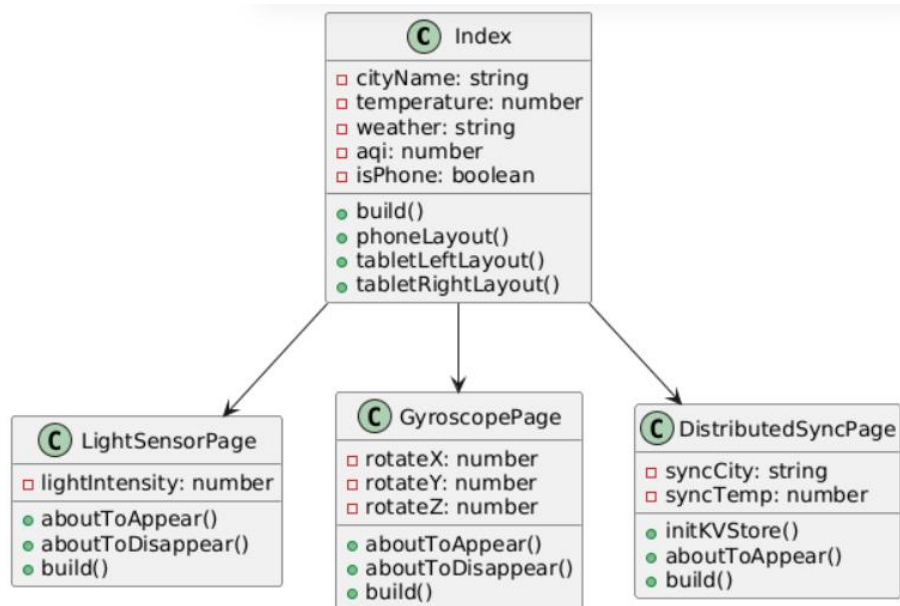
```

class GyroscopePage {
    - rotateX: number
    - rotateY: number
    - rotateZ: number
    + aboutToAppear()
    + aboutToDisappear()
    + build()
}

class DistributedSyncPage {
    - syncCity: string
    - syncTemp: number
    + initKVStore()
    + aboutToAppear()
    + build()
}

Index --> LightSensorPage
Index --> GyroscopePage
Index --> DistributedSyncPage
@enduml

```



类图解析

该图展示了项目核心页面组件及其结构关系，所有页面均采用 ArkTS 组件化开发，结构统一、易于维护。Index 作为应用入口，负责多端布局切换与页面导航；

LightSensorPage 和 GyroscopePage 分别负责光线、陀螺仪数据的监听与展示；

DistributedSyncPage 负责分布式数据的初始化、同步与展示，充分体现面向对象的设

计思想。

6.5.3 功能交互顺序图

PlantUML 代码

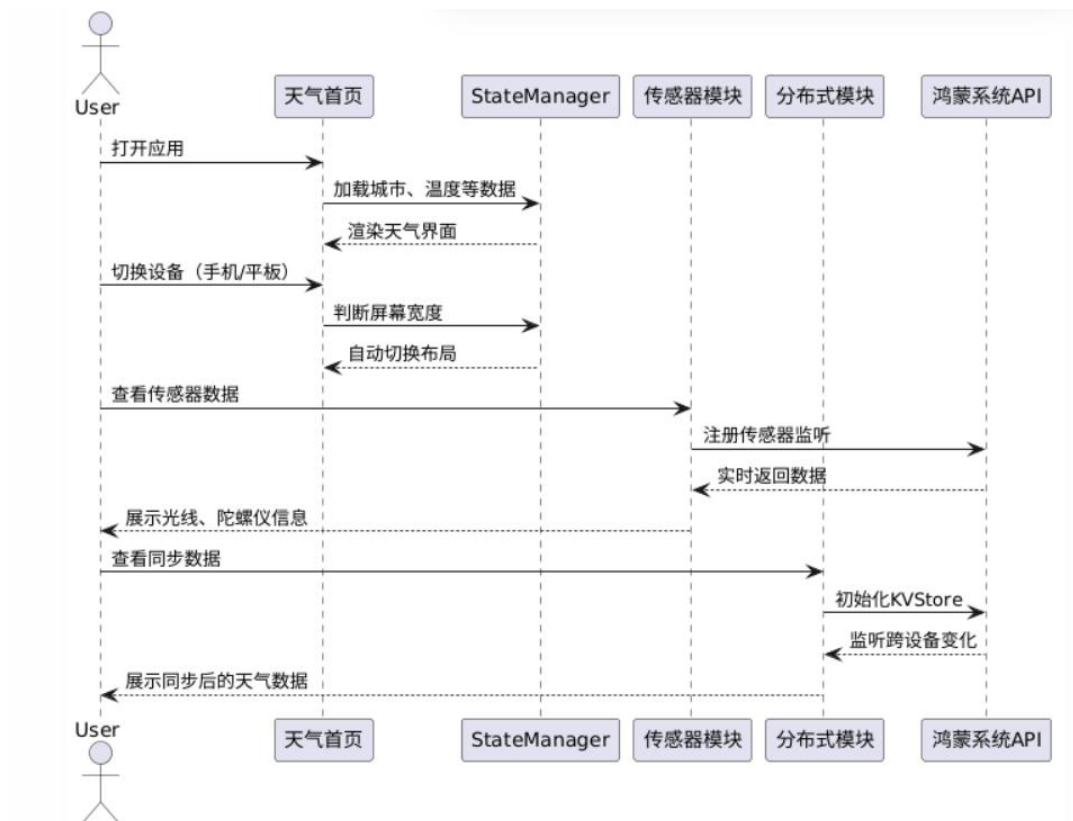
```
puml
@startuml 功能交互流程顺序图
actor User
participant 天气首页
participant StateManager
participant 传感器模块
participant 分布式模块
participant 鸿蒙系统 API

User -> 天气首页: 打开应用
天气首页 -> StateManager: 加载城市、温度等数据
StateManager --> 天气首页: 渲染天气界面

User -> 天气首页: 切换设备（手机/平板）
天气首页 -> StateManager: 判断屏幕宽度
StateManager --> 天气首页: 自动切换布局

User -> 传感器模块: 查看传感器数据
传感器模块 -> 鸿蒙系统 API: 注册传感器监听
鸿蒙系统 API --> 传感器模块: 实时返回数据
传感器模块 --> User: 展示光线、陀螺仪信息

User -> 分布式模块: 查看同步数据
分布式模块 -> 鸿蒙系统 API: 初始化 KVStore
鸿蒙系统 API --> 分布式模块: 监听跨设备变化
分布式模块 --> User: 展示同步后的天气数据
@enduml
```

顺序图解析

该图完整描述了用户从启动应用到完成核心功能的交互流程，涵盖天气页面加载、多端适配切换、传感器数据采集、分布式数据同步四大核心场景，清晰展示了用户、UI 页面、业务模块与鸿蒙系统 API 之间的调用时序和数据流向，便于理解应用整体运行逻辑，也为后续调试排查问题提供了清晰的流程参考。

七、技术对比与分析

7.1 核心技术特点

- **Maul 组件库**：简化多端适配开发，组件自带间距、安全区适配，无需手动编写适配逻辑，提升开发效率。
- **ArkTS 声明式 UI**：代码简洁，数据绑定灵活，可快速实现 UI 与数据的联动，便于调试与维护。
- **鸿蒙分布式能力**：跨设备数据同步便捷，无需复杂配置，依托 KVStore 即可实现多端数据一致性。
- **传感器 API**：调用简单，数据采集实时，可快速集成到多端应用中，丰富应用交互场景。

7.2 适用场景分析

本实验开发的智慧天气应用，基于 **Maul** 技术栈，适用于鸿蒙多端场景，可拓展至物联网设备、移动终端、平板等多设备，核心优势的是“一套代码、多端运行”，降低开发与维护成本，适合中小型多端应用开发。

八、实验总结

8.1 技术收获

1. 掌握了鸿蒙多端应用的开发流程，熟悉了 **ArkTS** 声明式开发模式与 **Maul** 组件库的使用方法。
2. 掌握了多端适配的核心原理，能够使用 **Maul** 组件实现手机、平板的自适应布局切换。
3. 掌握了鸿蒙传感器 **API** 的调用方法，能够实现光线、陀螺仪数据的采集与展示，理解了资源释放的重要性。
4. 掌握了鸿蒙分布式数据同步机制，能够实现跨设备数据实时同步，理解了分布式 **KVStore** 的使用流程。
5. 学会了使用 **AI** 工具（**TRAE**）辅助生成代码，提升开发效率，掌握了代码优化与异常处理的基本方法。

8.2 开发建议

1. 多端适配：开发前明确设备屏幕尺寸，利用 **Maul** 组件的适配能力，减少手动适配代码，提升开发效率。
2. 传感器开发：封装传感器监听与销毁的通用方法，便于复用，避免资源泄漏。
3. 分布式开发：提前确认设备环境（账号、网络），做好异常处理，避免同步失败导致的功能异常。
4. 代码规范：编写代码时添加详细注释，规范变量命名，提升代码可维护性与可读性。

8.3 后续学习建议

1. 深入学习 **Maul** 组件库的高级用法，实现更复杂的多端适配场景（如折叠屏适配）。
2. 学习鸿蒙分布式高级特性，实现更复杂的跨设备协同功能（如多设备联动操作）。
3. 掌握状态管理高级用法，结合 **@Link**、**@Provide** 等装饰器，实现复杂场景的数据

联动。

4. 学习应用打包、发布流程，了解鸿蒙应用的上线规范与审核要求。
5. 深入研究传感器的高级应用，结合物联网场景，实现更丰富的交互效果。